Document 1: Circuit

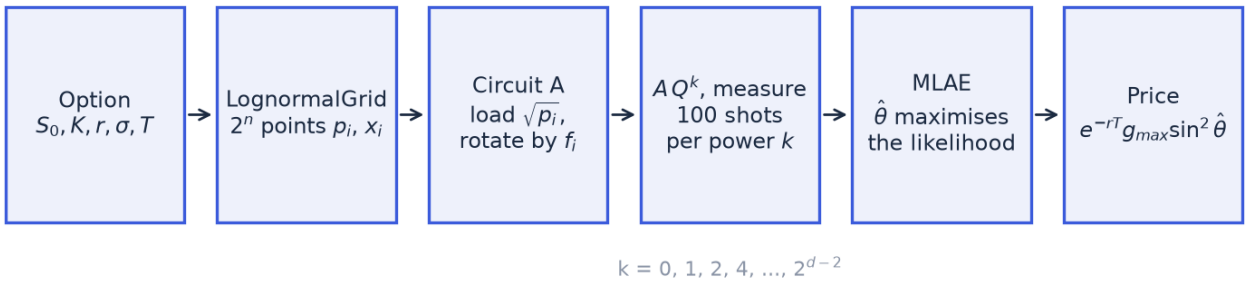
Kit: qae_european 1.0.0 · **Scope:** what the circuit computes, how, with which inputs and outputs, and how to reproduce every number in the kit.

Companion documents: Document 2: Benchmarks · Document 3: Tests · User guide.

---

1. Purpose

The kit prices a European call under Black-Scholes with **amplitude estimation** on a quantum circuit simulator. The aim is not to price this option (it has a closed form) but to provide the smallest complete example of the method: loading a distribution, encoding a payoff, estimating an amplitude, and validating the result against classical references.



2. Model and target

Under Black-Scholes, the terminal price is lognormal under the risk-neutral measure:

ln S_T ~ N(μ, s²), μ = ln S₀ + (r − σ²/2) T, s = σ √T.

The target is the call price C = e^(−rT) E[max(S_T − K, 0)]. The underlying pays no dividend. Inputs are validated in `Option` (`option.py`): S₀, K, σ, T must be positive and finite; r must be finite and may be negative.

3. Discretisation (`grid.py`)

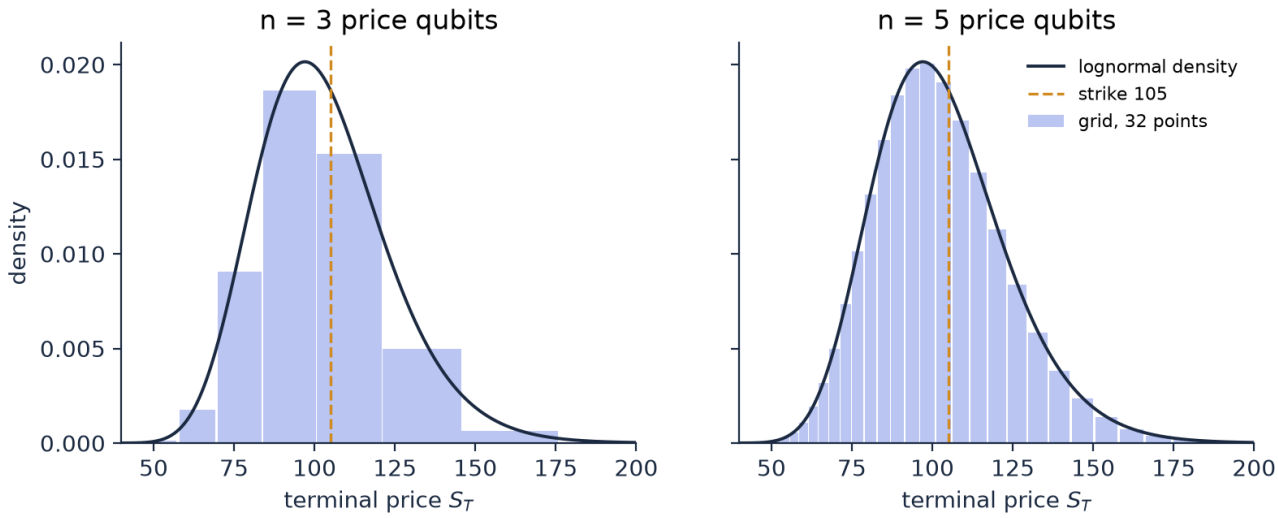
A circuit on n qubits holds 2^n basis states, so the distribution is discretised on 2^n points.

1. **Range.** The log-price interval [μ − w·s, μ + w·s] is cut into 2^n bins of equal width in log-price, with w = 4 by default (`width`).
2. **Probabilities.** Bin i carries p_i = Φ(z_{i+1}) − Φ(z_i), with z the standardised edges. The probabilities are renormalised to sum to 1. The mass outside the range, $2\Phi(-4) \approx 6.3 \cdot 10^{-5}$, is spread proportionally over the bins.
3. **Representative prices.** Bin i is represented by the conditional mean of S_T in the bin, which has a closed form for a lognormal:

   x_i = E[S_T | S_T in bin i] = e^(μ + s²/2) · [Φ(z_{i+1} − s) − Φ(z_i − s)] / p_i.

   Using conditional means rather than bin midpoints makes the discretised forward match the true forward, up to the truncated mass.

The **grid price** is P_grid = e^(−rT) Σ_i p_i max(x_i − K, 0). It is the exact target of the circuit. Its gap to Black-Scholes is the **discretisation bias**, which depends only on n, w and the option, not on any sampling. The bias falls as n grows.



Reference option of the guide: $S_0 = 100$, $K = 105$, $r = 3\%$, $\sigma = 20\%$, $T = 1$.

4. The circuit A (`quantum.py`)

The circuit acts on **n + 1 qubits**:

- qubits 0 to n − 1: the **price register**. Basis state |i⟩ stands for grid point i, with qubit 0 the least significant bit (Qiskit's convention).
- qubit n: the **objective qubit**.

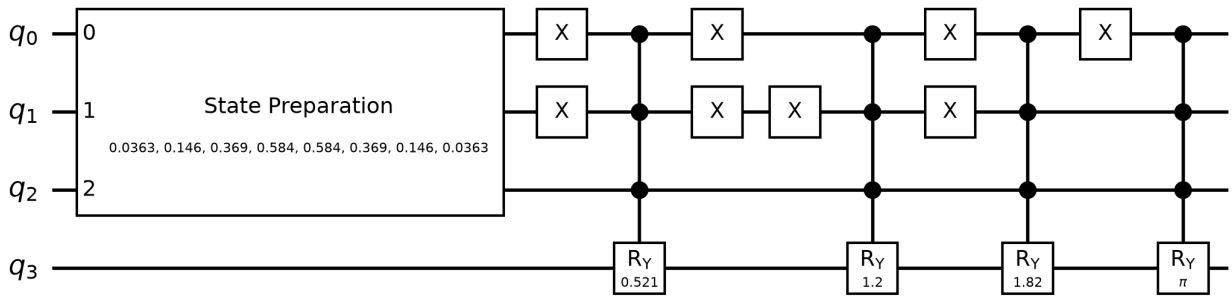
Block 1: load the distribution. `StatePreparation` prepares Σ_i √p_i |i⟩ on the price register. This is exact loading: Qiskit synthesises a circuit for an arbitrary state.

Block 2: encode the payoff. With g_i = max(x_i − K, 0) and g_max = max_i g_i, the normalised payoffs are f_i = g_i / g_max, in [0, 1]. For each grid point with f_i > 0, a rotation RY(θ_i) with θ_i = 2 arcsin(√f_i) is applied to the objective qubit, controlled on the price register being exactly |i⟩. The control on |i⟩ is built with X gates on the price qubits where bit i is 0, before and after a rotation controlled on all n price qubits. Grid points out of the money get no rotation.

After A, the state is Σ_i √p_i |i⟩ ⊗ (√(1 − f_i) |0⟩ + √f_i |1⟩). The probability of measuring the objective qubit in state 1 is therefore

a = Σ_i p_i f_i = E_grid[g] / g_max, and P_grid = e^(−rT) · g_max · a.

The kit writes a = sin²θ with θ in [0, π/2]. If every grid point is out of the money (g_max = 0), the option is **worthless on the grid**: the price is exactly 0 and nothing is estimated.



Four grid points are in the money, so A has four controlled rotations. The last one is RY(π), since f = 1 at the largest payoff.

5. The Grover operator Q

$Q = A\ S_0\ A\dagger\ S_\chi$, up to a global phase of π, where:

- S_χ applies a phase −1 to states with the objective qubit in 1: a Z gate on qubit n.
- S_0 applies a phase −1 to $|0\ldots0\rangle$. It is built as X on all qubits, a multi-controlled Z (H, multi-controlled X, H on qubit n), then X on all qubits.

Q rotates the state by 2θ in the plane spanned by the "good" and "bad" components of $A|0\rangle$. After A and k applications of Q:

$P(\text{objective} = 1) = \sin^2((2k + 1)\theta)$.

The code sets the global phase of Q explicitly (`global_phase += π`). It has no effect on measured probabilities, but it makes Q equal to the textbook operator $-A\ S_0\ A\dagger\ S_\chi$, which matters if Q is ever used under control.

6. Maximum-likelihood amplitude estimation (`estimation.py`)

Phase estimation would need many controlled applications of Q. MLAE (Suzuki et al., 2020) uses only plain circuits $A\ Q^k$ and classical post-processing.

Ladder. A ladder of depth d uses the Grover powers $k \in \{0, 1, 2, 4, \ldots, 2^{(d-2)}\}$: d powers, the largest $2^{(d-2)}$. Each circuit $A\ Q^k$ is measured 100 times (`shots`).

Cost. One measurement after k steps applies A once, then Q k times. Each Q contains A and A†. The kit counts this as 2k + 1 applications of A (A† is counted as one application). The total cost of a ladder is

$N = \text{shots} \times \Sigma_k\ (2k + 1)$.

With 100 shots, depths 4 to 8 give N = 1,800, 3,500, 6,800, 13,300 and 26,200 calls.

Estimator. With h_k ones out of `shots` at power k, θ maximises the log-likelihood

$L(\theta) = \Sigma_k\ [\ h_k \ln \sin^2((2k + 1)\theta) + (\text{shots} - h_k) \ln \cos^2((2k + 1)\theta)\ ]$.

It is maximised in two steps:

1. a grid search over equally spaced points on [0, π/2]: at least 4,001, and at least 200 per period of the most oscillating term ($100 \times m_{max} + 1$ points, with $m_{max} = 2k_{max} + 1$);
2. a bounded scalar refinement (`scipy.optimize.minimize_scalar`, tolerance 10^{-12}) within one grid step of each of the **5 highest local maxima** of the grid. The best refined point is kept.

Probabilities are clipped to $[10^{-15}, 1 - 10^{-15}]$ to avoid log(0). The price estimate is $e^{(-rT)}\ g_{max}\ \sin^2\hat{\theta}$.

Why several candidates: when two peaks of the likelihood are nearly tied, the grid can rank them in the wrong order, because it samples each peak slightly off its top. Refining only the best grid point would then return the lower peak. An earlier version did exactly that. An internal review found it failed in about 1 draw in 200 at depth 8 and in every draw at depth 14 on the widest scenario. It was corrected before the campaign was re-run (Document 3, section 1.9). A unit test now compares the estimator with a 400,001-point grid at depths 8 and 12, and `docs/checks.py` does the same for every campaign configuration with a mode error. `Ladder.exponential` accepts depths up to 16.

Known limitation: mode errors. L(θ) has several local maxima. The higher powers make it oscillate quickly, while the low powers, which only see a few shots each, are what tell those maxima apart. Occasionally the noise in the low-power counts makes a wrong local maximum the highest one. The estimate then lands about $\pi/(2k_{max} + 1)$ away from the true θ. Such errors are rare but large.

This is a property of MLAE with an exponential ladder, not of the optimiser: the estimate is the true global maximum of the likelihood (`docs/checks.py`, results in Document 3, section 2.9). The campaign keeps the published method unchanged, uses 50 draws per configuration, and records the share of draws affected (Document 3, sections 1.5 and 1.9).

How mode errors are counted. A draw counts as a mode error when $|\hat{\theta} - \theta| > \pi/(2\ m_max)$, half the period of the most oscillating term. Two consequences:

- The threshold is wide for short ladders ($\pi/10$ at depth 4). The absence of mode errors at low depth is partly a consequence of the definition.
- A wrong peak lying closer to the true θ than the threshold would not be counted.

7. Simulating the measurements

Two backends give the measurement probabilities. On a noise-free simulator they are mathematically identical:

Backend	How	Used for
`statevector`	Builds A and Q in Qiskit and evolves the exact statevector through A Q^k, then reads P(objective = 1).	Verification, the user guide
`born`	Takes θ from the exact statevector of A, then uses $P_k = \sin^2((2k + 1)\theta)$.	The campaign (2.31 million amplitude-estimation estimates)

In both cases, the measurement counts are then drawn as $h_k \sim \text{Binomial}(shots, P_k)$. This is exactly what a noise-free sampler would return.

The `born` backend relies on the identity in section 5. It is checked by full statevector simulation on 20 scenarios drawn at random, for every number of qubits where the option is not worthless (`qae-european verify`, hypothesis H0-b of Document 3):

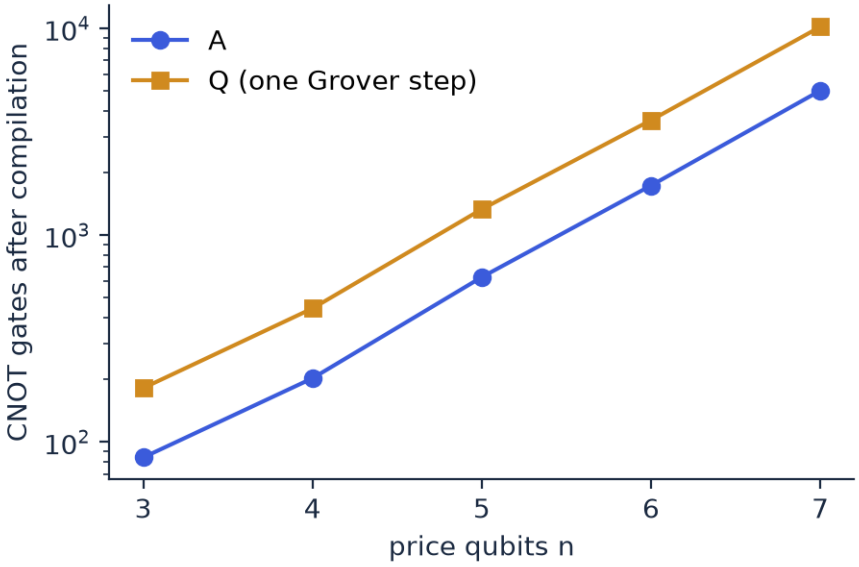
- for $n \leq 5$, powers up to 64, which covers the full ladder of depth 8;
- for $n = 6$ and 7, powers up to 8, because simulation cost grows with the depth of A Q^k; plus the full ladder (powers up to 64) on the first of the 20 scenarios, so the largest circuits are also checked at the largest power the campaign uses.

The identity holds for any k: it follows from Q being a rotation. Document 3 reports the outcome.

8. Circuit size

Compiled to a hardware-like gate set {CX, RZ, SX, X} with Qiskit's transpiler (optimisation level 1, seed 0), for the reference option:

Price qubits n	Qubits	CNOTs in A	Depth of A	CNOTs in Q	Depth of Q
3	4	84	290	182	606
4	5	203	387	442	852
5	6	626	1,037	1,334	2,178
6	7	1,737	2,548	3,596	5,230
7	8	5,000	12,155	10,170	24,491



The count roughly triples per added qubit, i.e. it grows exponentially. Both blocks are to blame:

- exact state preparation;
- one multi-controlled rotation per in-the-money point.

At depth 8, one measurement at k = 64 applies Q 64 times: about 85,000 CNOTs for n = 5. That is far beyond today's noisy hardware, which is why the kit runs on a simulator. Efficient loading is the open problem for practical use (Vazquez and Woerner, 2021; Stamatopoulos and Zeng, 2024).

9. Inputs and outputs

Inputs

Input	Where	Type and range	Default
spot, strike, vol, maturity	`Option`	float > 0	100, 105, 0.20, 1.0
rate	`Option`	float, may be negative	0.03
n_qubits	`LognormalGrid`	integer, 1 to 16	— (3 to 7 in the campaign)
width	`LognormalGrid`	float > 0, standard deviations	4.0
depth, shots	`Ladder.exponential`	integer ≥ 1	— , 100
backend	`MaximumLikelihoodAE`	`"born"` or `"statevector"`	`"born"`
seed / rng	all random methods	`numpy.random.Generator`	—

Invalid inputs raise `ValueError`. The kit uses **no external data**: every input is a contract or market parameter given by the user, or generated from the fixed scenario grid of the campaign.

Outputs

- `qae-european price`: JSON on standard output with the Black-Scholes, grid, exact-circuit, amplitude-estimation, Monte Carlo and quasi-Monte Carlo prices, and the number of calls. Prices are in the currency of the spot, discounted to today.
- `qae-european verify`: `results/verification.json`, one record per (scenario, number of qubits): the largest power simulated, the largest absolute difference against $\sin^2((2k + 1)\theta)$, and the circuit-against-grid price difference.
- `qae-european campaign`: `results/campaign.csv`, one row per (scenario, number of qubits, ladder depth). The columns are defined at the top of `src/qae_european/campaign.py` and in Document 3, section 1.5. Floats are written at full double precision.

10. Environment and reproducibility

Item	Value
Python	3.13.17
Qiskit	2.6.1
NumPy	2.5.4
SciPy	1.18.1
Full pinned list	`requirements-lock.txt`, frozen in a clean virtual environment
Operating system	Linux (x86-64, glibc 2.39)
CPU	2-core Intel Xeon, 2.8 GHz
Quantum hardware	None: local noise-free simulation (Qiskit `Statevector`)
Install	`pip install -r requirements-lock.txt` then `pip install --no-deps -e .` (user guide, section 1)

Seeds.

- The campaign derives every random stream from a master seed (20261103) and the indices of the scenario, number of qubits, ladder depth and method, through `numpy.random.SeedSequence` (`campaign.rng_for`). Any row can be recomputed on its own (user guide, section 5).
- The verification draws its 20 scenarios with the same master seed.
- The transpiler in section 8 uses seed 0.

Determinism. In the frozen environment, results are bit-for-bit reproducible:

- **Parallel runs:** they don't change any value. Rows may appear in a different order in the CSV, because each scenario's streams depend only on its own indices.
- **Other library versions:** they may change the last digits. The order of random draws is fixed by NumPy's `Generator` and SciPy's Sobol implementation, which are stable within a major version but not guaranteed across them.

Commands (from `kits/qae_european`):

```
pytest tests                                    # unit tests
qae-european verify                             # H0-b
qae-european campaign --processes 2             # Document 3 campaign
python docs/make_figures.py                     # figures of this document (needs the docs extra)
python docs/analyse.py                          # Document 3: criteria, summary.json and figures
python docs/checks.py                          # results/checks.json: state-preparation accuracy, MLAE
 ↪  global maximum
```

11. Limitations

- **Exact loading does not scale** (section 8). The kit demonstrates the method, not a practical pricer.
- **Numerical precision of the synthesis.** Qiskit's synthesis of the state preparation adds a floating-point error that grows with the number of qubits: about 10^{-13} at n = 5, 10^{-11} at n = 6 and 10^{-9} to 10^{-10} at n = 7 on the loaded probabilities (`docs/checks.py`). The circuit price then matches the grid price to a relative 10^{-8}, not to machine precision. This is far below any statistical error, but it matters for exact-equality tests (Document 3, section 2.4).
- **Noise-free simulation.** No hardware noise and no error mitigation. Real hardware would add errors that grow with circuit depth.
- **The error is scaled by the largest payoff.** The price is read as e^(−rT) g_max a. With high volatility or long maturity, g_max is far above the price: on the 5-year, 60%-volatility at-the-money scenario, g_max ≈ 7,100 for a price of about 49. The amplitude a is then small, and its error is magnified in the price. A narrower `width` reduces g_max but increases the truncation bias. The default, 4 standard deviations, is a common choice and is not tuned per scenario.
- **Mode errors of MLAE** (section 6).

- **Call counts are not run times.** On a fault-tolerant machine, one application of A takes far longer than one Monte Carlo path on a classical processor. Published estimates of where a practical advantage could appear are in Chakrabarti et al. (2021) and Stamatopoulos and Zeng (2024).
- **One product.** The construction applies to any payoff of S_T that is non-negative on the grid (user guide, section 7). Path-dependent payoffs need a register per time step and are out of scope.

References

- Suzuki, Uno, Raymond, Tanaka, Onodera, Yamamoto, *Amplitude estimation without phase estimation*, Quantum Information Processing 19, 75 (2020). arXiv:1904.10246
- Stamatopoulos, Egger, Sun, Zoufal, Iten, Shen, Woerner, *Option pricing using quantum computers*, Quantum 4, 291 (2020). arXiv:1905.02666
- Brassard, Høyer, Mosca, Tapp, *Quantum amplitude amplification and estimation*, Contemporary Mathematics 305 (2002). arXiv:quant-ph/0005055
- Chakrabarti, Krishnakumar, Mazzola, Stamatopoulos, Woerner, Zeng, *A threshold for quantum advantage in derivative pricing*, Quantum 5, 463 (2021). arXiv:2012.03819
- Vazquez, Woerner, *Efficient state preparation for quantum amplitude estimation*, Physical Review Applied 15, 034027 (2021). arXiv:2005.07711
- Stamatopoulos, Zeng, *Derivative pricing using quantum signal processing*, Quantum 8, 1322 (2024). arXiv:2307.14310